



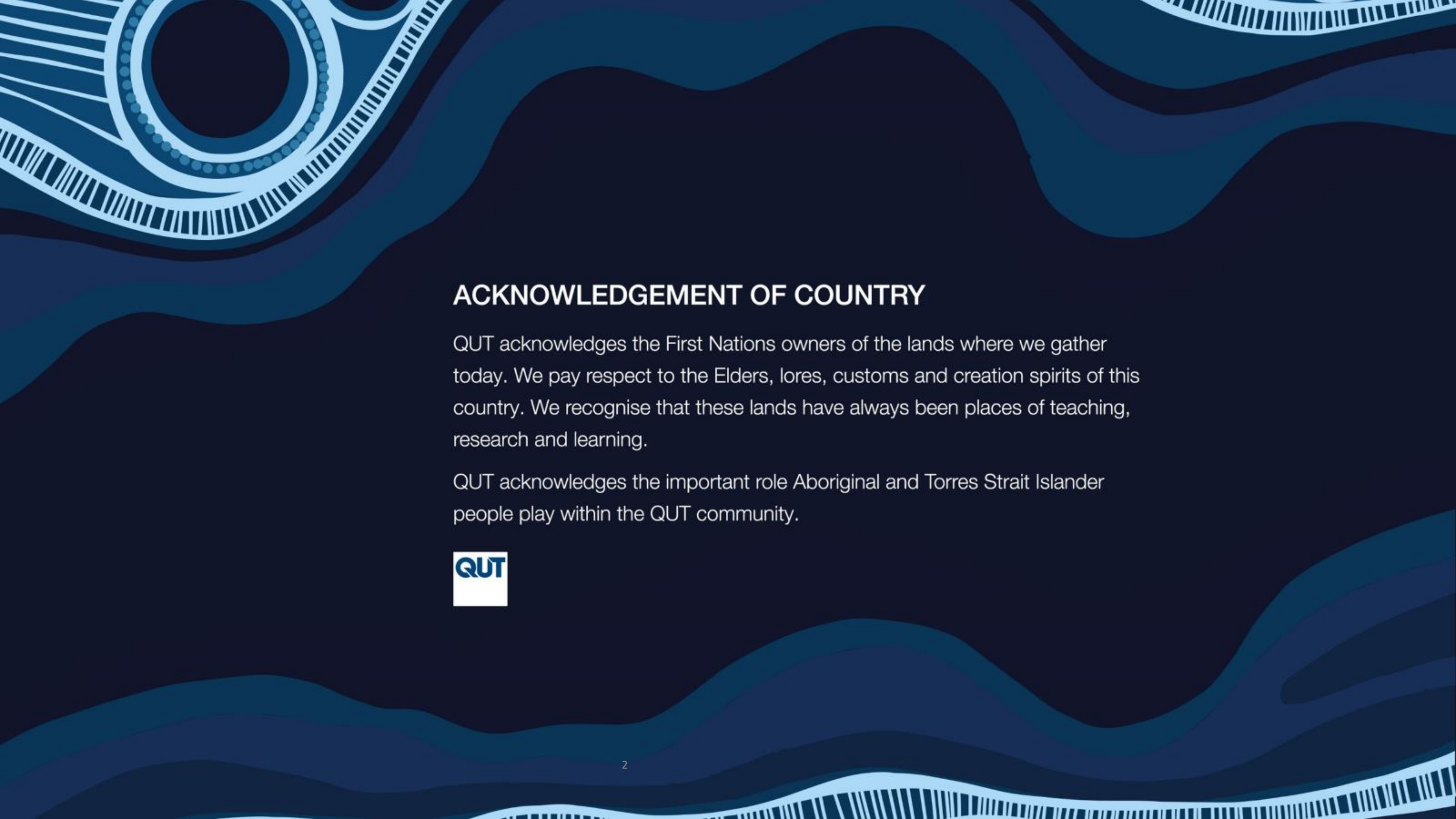
ARC Training Centre for Behavioural
Insights for Technology Adoption

SYNTHETIC DATA GENERATION

A QUICK GUIDE

ARIAN MASHHADY





ACKNOWLEDGEMENT OF COUNTRY

QUT acknowledges the First Nations owners of the lands where we gather today. We pay respect to the Elders, lores, customs and creation spirits of this country. We recognise that these lands have always been places of teaching, research and learning.

QUT acknowledges the important role Aboriginal and Torres Strait Islander people play within the QUT community.





WHAT IS SYNTHETIC DATA

- Artificially generated data.
- It is not a one-to-one match with real-world observations.

WHY DOES SYNTHETIC DATA MATTER

- Protecting Sensitive Information.
- We simply do not have enough data.
- We want to correct bias in our data
- Some examples:
 - Example 1: Cloud AI Services.
 - Example 2: System Prototyping.
 - Example 3: Health Economics Research.



WHAT IS SYNTHETIC DATA VAULT

- Developed by MIT.
- Synthetic Data Vault is an open-source Python library.
- Enterprise tier available for special use cases and early access.
- Maintains fidelity and privacy.
- Provides various quality and privacy tests.

WHAT IS SYNTHETIC DATA VAULT

- Core capabilities:
 - Privacy protection
 - Generated data is not a one-to-one match.
 - Statistical fidelity
 - Maintains distributions, preserves the relationship between variables (sort of).
- Various quality and privacy tests for different use cases
 - Various testing tools through SDMetrics to validate synthetic data.

TYPES OF DATA SDV CAN GENERATE

- Single table data: each row represents one entity.

- Example:

customer_id	age	income	gender
1	34	75,000	F
2	45	90,000	M
3	28	55,000	F

- Multi-table data: data spread across multiple related tables with connections between them.

- Example:

household_id	income	region
1	90,000	QLD
2	60,000	NSW



person_id	household_id	age	education
1	1	40	Bachelor
2	1	12	School
3	2	33	Diploma

- Notice how household_id links people to their households, creating relationships across tables.

TYPES OF DATA SDV CAN GENERATE

- Sequential Data: captures sequences of events where order matters.
 - Example: Job seeker journey.
 - Modelling how job seekers progress through their search:
 - Apply to jobs.
 - Attend interviews.
 - Receive offers.
 - Accept/reject offers.

person_id	step	action
1	1	apply
1	2	interview
1	3	accept
2	1	apply
2	2	reject
3	1	apply
3	2	interview
3	3	reject



GETTING STARTED WITH SDV

PREREQUISITES, DATA PREPARATION, TRAINING, GENERATING, AND
EVALUATING

- **Prerequisites**
 - **A working IDE or text editor**
 - Jupyter Notebook (Suggested for interactive exploration)
 - PyCharm
 - Visual Studio Code
 - Or any Python-compatible environment

- **Prerequisites**

Installation:



#1. VS code and PyCharm

```
pip install sdv
```

#2. Conda environment

Create a new environment with Python 3.10

```
conda create -n sdv-env python=3.10 -y
```

Activate the environment

```
conda activate sdv-env
```

Install SDV

```
pip install sdv
```




DATA PREPARATION



DATA PREPARATION

- Data Requirements
 - SDV expects two objects:
 - A pandas DataFrame: this is your real data.
 - Metadata: Metadata is a comprehensive description of the dataset you want to synthesise.
 - categorical, numerical, datetime, etc.

DATA PREPARATION

- Metadata example:

table: hotels

hotel_id	city	rating
HID_000	Boston	4.8
HID_001	Boston	4.1
HID_002	San Francisco	3.8
⋮	⋮	⋮

table: guests

guest_email	hotel_id	checkin_date	checkout_date	room_type
awolf@phillips.com	HID_001	27 Dec 2020	28 Dec 2020	BASIC
harriskathleen@goodwin.com	HID_001	30 Dec 2020	02 Jan 2021	DELUXE
fclark@mayo.com	HID_000	17 Sep 2020	19 Sep 2020	BASIC
brenda70@miller.com	HID_002	05 Apr 2020	10 Apr 2020	SUITE
⋮	⋮	⋮	⋮	⋮

```
{
  "METADATA_SPEC_VERSION": "V1",
  "tables": {
    "hotels": {
      "primary_key": "hotel_id",
      "columns": {
        "hotel_id": { "sdtype": "id", "regex_format": "HID_[0-9]{3}" },
        "city": { "sdtype": "categorical" },
        "rating": { "sdtype": "numerical" }
      },
      "column_relationships": []
    },
    "guests": {
      "primary_key": "guest_email",
      "columns": {
        "guest_email": { "sdtype": "email" },
        "hotel_id": { "sdtype": "id", "regex_format": "HID_[0-9]{3}" },
        "checkin_date": { "sdtype": "datetime", "datetime_format": "%d" },
        "checkout_date": { "sdtype": "datetime", "datetime_format": "%d" },
        "room_type": { "sdtype": "categorical" }
      },
      "column_relationships": []
    }
  },
  "relationships": [{
    "parent_table_name": "hotels",
    "parent_primary_key": "hotel_id",
    "child_table_name": "guests",
    "child_foreign_key": "hotel_id"
  }]
}
```

DATA PREPARATION

- Automatic Metadata Detection
 - SDV provides auto-detection capabilities.
- Using auto-detection:
 - Detect
 - Inspect
 - Validate
 - Save (if needed)

```
from sdv.metadata import Metadata

# Step 1: Auto-detect metadata based on your data

metadata = Metadata.detect_from_dataframes(
    data={
        'hotels': hotels_dataframe,
        'guests': guests_dataframe
    })

# Step 2: Carefully inspect and update your metadata

metadata.visualize()

# Step 3: Validate that everything is correct

metadata.validate()

# Make manual corrections where needed

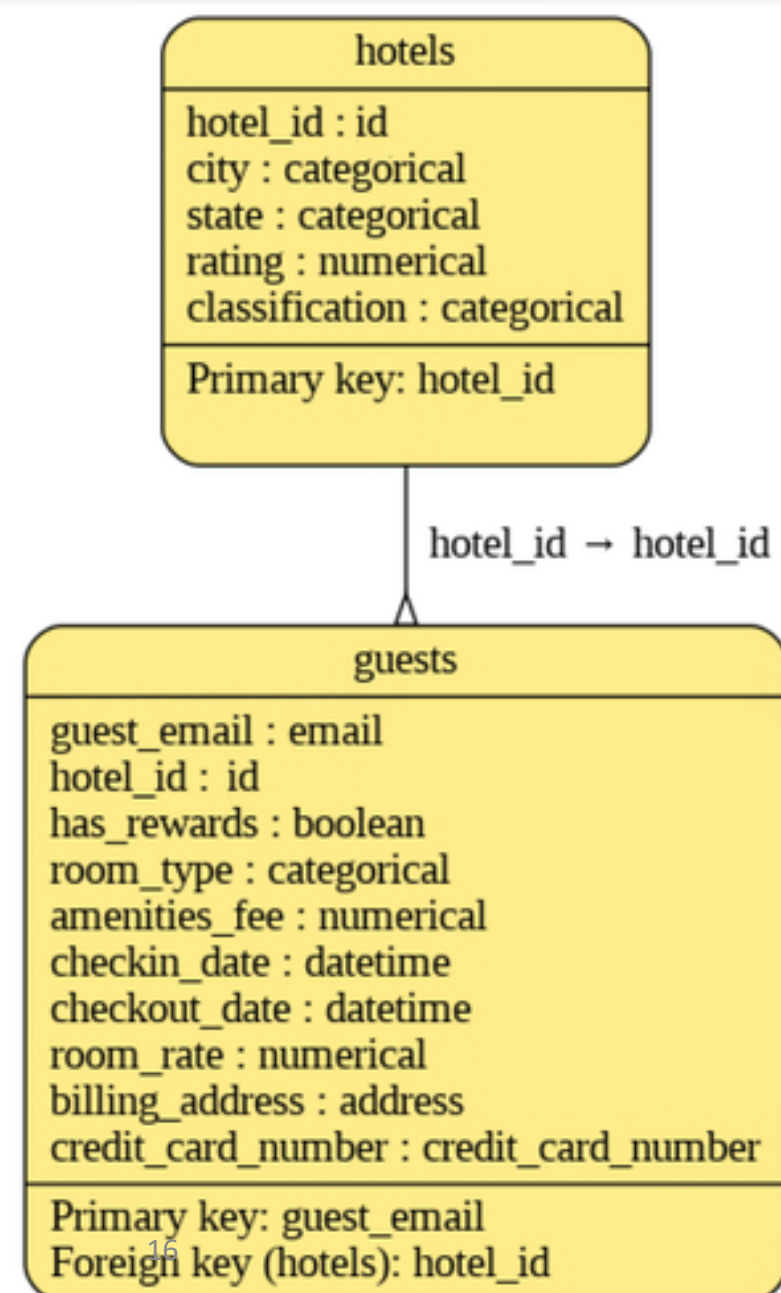
metadata.update_column(
    column_name='room_type',
    sdtype='categorical',
    table_name='guests'
)

# Step 4: Save it for future use

metadata.save_to_json('my_final_metadata.json')
```


DATA PREPARATION

- When you visualize:



- Demo data
 - SDV includes numerous demo datasets
 - How to use:

```
from sdv.datasets.demo import download_demo
from sdv.datasets.demo import get_available_demos

get_available_demos(modality='single_table') #what demos are available

data, metadata = download_demo(
    modality='single_table',
    dataset_name='fake_hotel_guests'
) #download demos
```



MODELING AND TRAINING DATA



MAIN FOCUS: SINGLE TABLE DATA

- Represents the most common use case.
- Time constraint.

UNDERSTANDING SYNTHESISERS

- Synthesiser:
 - learns the statistical properties of your real data.
 - Using what it learned, it generates new synthetic records that maintain those same properties.
- Some of the important synthesisers:
 - The Gaussian Copula Synthesiser
 - CTGAN Synthesizer
 - TVAE Synthesizer



The Gaussian Copula Synthesiser

WHAT IS A COPULA

- A copula is a mathematical tool that describes how random variables relate to each other.
 - Copula has two aspects:
 - Marginal Distributions: How each individual variable behaves on its own.
 - Income might follow a right-skewed distribution.
 - Age might follow a more uniform or normal distribution.
 - Dependence Structure: How variables move together.
 - Higher education levels typically correlate with higher income

WHY GUASSIAN COPULA

- A Gaussian Copula assumes :
 - Dependence structure between variables resembles a multivariate normal (bell curve) distribution.
 - Each individual variable can still have its own distinct shape.
- Even if your income data is highly skewed and your age data is uniformly distributed, the Gaussian Copula can still accurately model how income and age correlate with each other.

HOW THE GAUSSIAN COPULA SYNTHESISER WORKS

- The Gaussian copula function is formally defined as:

$$C_{\Sigma}^{Gauss}(U_1, \dots, U_n) = \Phi_{\Sigma}(\Phi^{-1}(U_1), \dots, \Phi^{-1}(U_n))$$

- Where Φ^{-1} maps uniform values to Gaussian values, and Φ_{Σ} is the multivariate normal CDF with correlation matrix Σ .

HOW THE GUASSIAN COPULA SYNTHESISER WORKS

- **Step 1:**
 - Transform each variable to a uniform scale between 0 and 1: using each variable's own cumulative distribution function (CDF).

- Example:

“What percentile is this income in our dataset?” That gives you a uniform value.

$$U_i = F_i(x_i)$$

- F is the empirical CDF of variable i. This gives us uniform values between 0 and 1.

HOW THE GUASSIAN COPULA SYNTHESISER WORKS

- **Step 2:**
 - Transform to Gaussian Space:
 - Apply an inverse normal (Gaussian) transformation to convert these uniform values into standard normal values (mean = 0, standard deviation = 1).
 - This creates a "copula space."

$$Z_i = \phi^{-1}(U_i)$$

- Where Φ^{-1} is the inverse of the standard normal CDF. Now all our variables are expressed in terms of standard normal distributions.

HOW THE GAUSSIAN COPULA SYNTHESISER WORKS

- **Step 3:**
 - In the Gaussian space, we calculate the correlation matrix Σ (Sigma)
 - Shows how all the transformed variables move together:

$$\Sigma = \text{corr}(Z_1, Z_2, \dots, Z_n)$$

- This correlation matrix is the key to preserving relationships in your synthetic data.

HOW THE GUASSIAN COPULA SYNTHESISER WORKS

- **Step 4:** Generate Independent Normal Variables

$$Z_{indep} = [z_1, z_2, \dots, z_n], \text{ where each } z_i \sim N(0,1)$$

- No relationship to each other yet.

HOW THE GUASSIAN COPULA SYNTHESISER WORKS

- **Step 5:** Introduce Correlation Using Cholesky Decomposition

- We use Cholesky decomposition of the correlation matrix Σ to introduce correlation:

$$\Sigma = LL^T$$

- L is a lower-triangular matrix. Sigma is positive semi-definite.
- We then multiply our independent variables by this matrix:

$$Z_{corr} = L \cdot Z_{indep}$$

- Original random normal variables are independent, but the Cholesky multiplication "mixes" them in precisely the right way so they follow the correlation structure observed in your real data.

HOW THE GUASSIAN COPULA SYNTHESIZER WORKS

- **Step 6: Map Back to Original Distributions**
 - We reverse the transformations to get back to your original data scale.
 - Transform each correlated Gaussian variable back to uniform scale:

$$U_i^{synthetic} = \Phi(Z_{corr}, i)$$

Invert the original marginal CDF to get synthetic data in the original scale.

$$X_i^{synthetic} = F_i^{(-1)}(U_i^{synthetic})$$

- What we end up with is synthetic data that maintains the original shape.
- Gaussian Copula Synthesizer uses these classic statistical methods to create synthetic data.

IMPLEMENTING THE GAUSSIAN COPULA SYNTHESIZER

- The Three-Step Process:

```
from sdv.single_table import GaussianCopulaSynthesizer

# Step 1: Create the synthesizer with your configuration

synthesizer = GaussianCopulaSynthesizer(
    metadata, # required: tells the synthesizer about your data structure
    enforce_min_max_values=True, # ensures synthetic values stay within original bounds
    enforce_rounding=False, # controls whether to round numerical values
    numerical_distributions={
        'amenities_fee': 'beta', # specify beta distribution for this variable
        'checkin_date': 'uniform' # specify uniform distribution for dates
    },
    default_distribution='norm' # use normal distribution for other variables by default
)

# Step 2: Train the synthesizer on your real data

synthesizer.fit(data)

# Step 3: Generate synthetic data

synthetic_data = synthesizer.sample(num_rows=10)
```


IMPLEMENTING THE GAUSSIAN COPULA SYNTHESISER

- The numerical_distributions parameter:
 - Specify which statistical distribution best fits each numerical column.
 - Default is beta. Fine in most situations, but sometimes the model might make a mistake and train the data with the wrong distribution.

```
numerical_distributions={  
    'amenities_fee': 'beta',  
    'checkin_date': 'uniform'  
}
```

- SDV supports: 'norm' 'beta', 'truncnorm', 'uniform', 'gamma' or 'gaussian_kde'

IMPLEMENTING THE GAUSSIAN COPULA SYNTHESISER

- Why change distribution from default?
 - Default is not guaranteed to work on every dataset. Set the distribution of important variables manually.
- How to check learned distributions?

```
synthesizer.get_learned_distributions()  
  
# output  
{  
  'amenities_fee': {  
    'distribution': 'beta',  
    'learned_parameters': { 'a': 2.22, 'b': 3.17, 'loc': 0.07, 'scale': 48.5 }  
  },  
  'checkin_date': {  
    ...  
  },  
  ...  
}
```

IMPLEMENTING THE GAUSSIAN COPULA SYNTHESIZER

- Do I need to train my synthesiser everytime? No.

```
synthesizer.save(  
    filepath='my_synthesizer.pkl'  
)
```

- Does it support categorical variables? Yes, SDV handles non-numerical columns through reversible data transforms:
 - categorical and other non-numerical data types are:
 - Transformed into numerical representations.
 - Processed through the copula.
 - Transformed back to their original types.

The CTGAN Synthesiser

UNDERSTANDING GENERATIVE ADVERSARIAL NETWORKS

- What is a “GAN”?
 - Generative Adversarial Network is a type of machine learning model.
 - Learns to generate synthetic data through an adversarial process.
- CTGAN is a specialised GAN and is designed for tabular data.
 - Handles skewed distributions and categorical variables.

UNDERSTANDING GENERATIVE ADVERSARIAL NETWORKS

- The Two-Network Architecture:
 - Two neural networks that compete against each other.
 - A game between two players, each trying to outsmart the other.
- The generator:
 - Generates synthetic records.
 - Learns and improves over time by trying to fool its opponent.
- The Discriminator:
 - Examines rows of data and determines whether each row is real.
 - Becomes increasingly skilled at spotting fake data.
- These two networks "compete" in a training loop.
- The Generator gets progressively better at fooling the Discriminator.
- The discriminator simultaneously gets better at detecting fake records.
- **Example:** Imagine Generator as a forger of money and Discriminator as a bank teller checking bills.

THE MATHEMATICS BEHIND GANs

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} \left[\log (1 - D(G(z))) \right]$$

- G (Generator): Creates fake/synthetic data
 - Input: random noise z .
 - Output: fake data $G(z)$ that tries to look real.
- D (Discriminator):
 - A classifier that outputs a probability.
 - Input: data (real or fake).
 - Output: $D(x) \in [0,1]$ = probability that input is real.
 - $D(x) \approx 1$ means "I think this is real" $D(x) \approx 0$ means "I think this is fake"
- The Game: Min-Max: The Discriminator wants to maximize $V(D, G)$ and Generator wants to minimize $V(D, G)$.

THE MATHEMATICS BEHIND GANs

- Expected value of $\log D(x)$ over real data:

$$E_{x \sim p_{data(x)}} [\log D(x)]$$

- $x \sim p_{data(x)}$ Sample x from the real data distribution.
- $D(x)$: Discriminator's output on real data.
- $\log D(x)$: Log probability that the discriminator assigns to real data.

- Expected value of $\log(1-D(G(z)))$ over generated data:

$$E_{z \sim p_{z(z)}} \left[\log \left(1 - D(G(z)) \right) \right]$$

- $z \sim p_{z(z)}$ Sample random noise z (e.g., from normal distribution).
- $G(z)$: Generator creates fake data from noise.
- $D(G(z))$: Discriminator's output on fake data.
- $\log(1-D(G(z)))$: Log probability that fake data is fake.

THE MATHEMATICS BEHIND GANs

- Discriminator's goal:
 - Make $D(x) \rightarrow 1$ for real data.
 - Make $D(G(z)) \rightarrow 0$ for fake data.
- Generator wants to minimise:

$$E_{z \sim p_z(z)} \left[\log \left(1 - D(G(z)) \right) \right]$$

- This happens when $D(G(z)) \rightarrow 1$
- Goal is to make Discriminator to think the fake data is real!

THE MATHEMATICS BEHIND GANs

- The Training Cycle:
 - **Fix G, train D:**
 - Show D real and fake samples
 - D learns to maximize $V(D,G)$
 - D gets better at spotting fakes
 - **Fix D, train G:**
 - G generates fake samples
 - G learns to minimize $V(D,G)$
 - G gets better at fooling D
 - **Repeat until equilibrium:**
 - Eventually D can't tell real from fake ($D(x) \approx 0.5$ everywhere)
 - G has learned to generate realistic data!

IMPLEMENTING THE CTGAN SYNTHESIZER

- Same pattern as the Gaussian Copula Synthesiser:

```
from sdv.single_table import CTGANSynthesizer

synthesizer = CTGANSynthesizer(
    metadata, # required: describes your data structure
    enforce_rounding=False, # whether to round numerical values
    epochs=500, # number of complete training cycles
    verbose=True # whether to print progress during training
)

synthesizer.fit(data) #train the model

synthetic_data = synthesizer.sample(num_rows=10) #generate data
```


IMPLEMENTING THE CTGAN SYNTHESISER

- Understanding Loss
 - In machine learning, loss is a measure of error.
- Discriminator Loss:
 - How bad the Discriminator is at detecting real versus fake data.
- Generator Loss :
 - How bad the Generator is at fooling the Discriminator.
- Equilibrium:
 - Generator and Discriminator compete in alternating steps until their losses stabilise.
 - Discriminator can only guess correctly about 50% of the time

IMPLEMENTING THE CTGAN SYNTHESIZER

- Understanding Epochs: The epochs parameter specifies how many times the model should process all your data during training.
- Using Verbose Mode: Setting verbose=True allows you to track the training process in real-time.
- GPU Acceleration: supports CUDA, so you can utilise your GPU (If available) for faster processing.
- Monitoring Training Progress:
 - visualize how well the training progressed:
 - plot the loss values at each epoch for both the generator and discriminator:

```
fig = synthesizer.get_loss_values_plot()  
fig.show()
```



Gaussian Copula versus CTGAN



Gaussian Copula versus CTGAN

- As a rule of thumb, if you don't have enough data (less than 1000 rows) use the Gaussian Copula Synthesiser.
- Datasets with at least several thousand rows, CTGAN becomes the better choice.
 - Especially if you have many categorical variables, skewed distributions, or complex non-linear relationships.



TVAE and COPULAGAN SYNTHESISERS

What is TVAE?

- We won't dive deeply into TVAE (Tabular Variational AutoEncoder) in this presentation.
- It is another option among SDV synthesisers.
- A more stable alternative to CTGAN.
- Data requirement stands between Gaussian copula and CTGAN.

What is The Copula GAN Synthesizer?

- uses a mix of classic, statistical methods and GAN-based deep learning methods to train a model and generate synthetic data.

Comparing all four Models

- GaussianCopula:
 - Datasets that are mostly numeric with simple correlations.
 - Quick, stable synthetic data generation when you don't have massive amounts of data or extremely complex patterns.
- CTGAN:
 - Shines with mixed data types.
 - Many categorical variables or distributions are skewed or imbalanced.
 - Needs more data to train effectively.
 - Requires careful monitoring and parameter tuning.
- TVAE:
 - Handles mixed data types with complex correlations.
 - More stable than CTGAN.
- CopulaGAN:
 - Provide benefits when you have data characteristics that fall between the sweet spots of the other methods. CopulaGAN gives you better marginals than pure copulas and better dependence than pure GANs, but it's more complex and computationally expensive than either approach alone.



CONSTRAINTS

Why constraints?

- Sometimes there are some hidden rules in your dataset.
- Example case:
 - Check_in date should be earlier than checkout date.
- A father's age needs to be greater than his children's ages by a biologically realistic margin.
- Here's an example of data I generated, forgetting to implement it for my own project :
 - "Sex: female; marital status: married (in a registered marriage); age: 71; non-resident children ages: 5, 32, 73; has grandchildren: yes, four"
 - 71-year-old person has a 73-year-old child !

SDV's Constraint Library

- SDV provides a comprehensive library of pre-defined constraints.
- They cover the most common scenarios.
- Some constraints:
 - FixedCombinations :
 - No shuffling is allowed beyond what's already observed in your original data.
 - Example:
 - In your real data, you might have entries like "Brisbane, Australia" or "New York, United States."
 - You do not need Brisbane, United States.
 - FixedIncrements:
 - All numerical values must be increments of a specified whole number.
 - Example:
 - In many organisations, salaries are set in increments of 1,000. You might have employees earning 45,000, 46,000, or 50,000, but never 45,723.50.

SDV's Constraint Library

- Inequality:
 - Value in one column must always be greater than (or less than) the value in another column.
 - The checkout_date must always be after the checkin_date.
- Range
 - The value in one column is bounded by the values in other columns.
- And more!

When Pre-Defined Constraints Aren't Enough

- Your research might have rules that don't fit any standard pattern.
- SDV allows you to design more advanced, custom constraints tailored to your exact requirements.
- It requires a good understanding of Python.
- For full tutorial:
 - [Tutorial](#)

- SDV's example:
 - [Full example](#)

```
# This is an example constraint as described in the "Program your own constraint" tutorial
# For the full tutorial, please see the link below:
# https://colab.research.google.com/drive/1BvUGkgYaf2ohMISqWZGrVTHgR1dwa0kV?usp=sharing
from sdv.cag import SingleTableProgrammableConstraint

class RoomRateCalculation(SingleTableProgrammableConstraint):
    """ The RoomRateCalculation constraint enforces that the
        total_rate = room_rate + amenities_fee """

    def transform(self, data):
        """ Preprocess the data by removing the total_rate column """
        preprocessed_data = data.drop(columns=['total_rate'])
        return preprocessed_data

    def get_updated_metadata(self, metadata):
        """ Preprocess the metadata by removing the total_rate column """
        new_metadata = metadata.copy()
        new_metadata.remove_column('total_rate')
        return new_metadata

    def reverse_transform(self, synthetic_data):
        """ Recreate the total rate column using the other columns """
        synthetic_data['total_rate'] = synthetic_data['room_rate'] + synthetic_data['amenities_fee']
        return synthetic_data

    def is_valid(self, data):
        """ Check that the total_rate = room_rate + amenities_fee for every row """
        column_sum = data['room_rate'] + data['amenities_fee']
        return (data['total_rate'] == column_sum)

class ColumnSum(SingleTableProgrammableConstraint):
    """ The ColumnSum constraint is a general constraint that enforces that the sum of various
        columns is equal to the final result column. You can supply the column names as parameters,
        allowing you to re-use this constraint in different datasets. """

    def __init__(self, columns_to_add, final_result_column):
        """ The user should provide:
            columns_to_add (list[str]): A list of column names that should be added up
            final_result_column (str): The name of the final result column
        """
        self.columns_to_add = columns_to_add
        self.final_result_column = final_result_column

    def transform(self, data):
        """ Preprocess the data by removing the final_result_column """
        preprocessed_data = data.drop(columns=[self.final_result_column])
        return preprocessed_data

    def get_updated_metadata(self, metadata):
        """ Preprocess the metadata by removing the final_result_column """
        new_metadata = metadata.copy()
        new_metadata.remove_column(self.final_result_column)
        return new_metadata

    def reverse_transform(self, synthetic_data):
        """ Recreate the final_result_column by adding the other columns """
        total_sum = synthetic_data[self.columns_to_add].sum(axis=1, skipna=True)
        synthetic_data[self.final_result_column] = total_sum
        return synthetic_data

    def is_valid(self, data):
        """ Validate that the sum of all the columns is equal to the final_result_column for every row """
        total_sum = data[self.columns_to_add].sum(axis=1, skipna=True)
        return (data[self.final_result_column] == total_sum)
```

Sampling: From Basic to Conditional

- You've trained your synthesiser.
- You're ready to generate synthetic data.
- Basic sampling:
 - Where you request a specified number of rows and the synthesiser generates records.
- Conditional Sampling:
 - You want more data with specific characteristics
 - Certain columns must have particular values.
 - You have rare events in your data that you want to oversample for testing purposes.
 - Synthesiser will only generate rows that meet these conditions.

Implementing Conditional Sampling

```
from sdv.sampling import Condition

# Step 1: Create Your Conditions

# We want 250 rows of guests who booked suites AND have rewards membership

suite_guests_with_rewards = Condition(
    num_rows=250, # How many rows we want
    column_values={'room_type': 'SUITE', 'has_rewards': True} # The conditions to meet
)

# Step 2: Sample Synthetic Data
# The synthesizer will generate only rows that match our conditions

synthetic_data = synthesizer.sample_from_conditions([suite_guests_with_rewards])

# the room_type column must equal 'SUITE'
# has_rewards column must be True
# The synthesizer then generates synthetic data that satisfies both conditions while still maintaining realistic values for all other columns.
```



Evaluation and Quality Assurance



HOW TO EVALUATE

- Why Evaluation Matters:
 - How good is this synthetic data?
- Two main tools:
 - SDV's standard library quick diagnostic.
 - Are all the required columns present?
 - Do the data types match?
 - SDMetrics for detailed evaluation.
 - Tests for statistical quality.
 - Tests for Data privacy.

HOW TO EVALUATE: SDV SIMPLE DIAGNOSTIC

- After generating new records, use the following:

```
from sdv.evaluation.single_table import run_diagnostic

diagnostic_report = run_diagnostic(
    real_data=real_data,
    synthetic_data=synthetic_data,
    metadata=metadata)
```

- Diagnostic tests **must** show 100 percent before continuing. If not, something is wrong, and you should not progress to the next step.
- If it is not 100 percent:

```
from sdv.evaluation.single_table import run_diagnostic

diagnostic_report.get_details(property_name='Data Validity')
```

HOW TO EVALUATE: SDV/SDMETRICS DATA QUALITY

- You can check for statistical quality:
 - Score is between 0 and 100.
 - Aim for above 80 percent.
 - Column shapes: statistical similarity between the real and synthetic data for single columns of data (marginal distribution of each column).
 - Column pair trends: bivariate distribution of the columns.

```
from sdv.evaluation.single_table import evaluate_quality

#using SDV

quality_report = evaluate_quality(
    real_data=real_data,
    synthetic_data=synthetic_data,
    metadata=metadata)

# using SDMetrics

from sdmetrics.reports.single_table import QualityReport

report = QualityReport()
report.generate(real_data, synthetic_data, metadata)
```


HOW TO EVALUATE: SDMETRICS

- SDMetrics is an open-source Python package:
 - Metric-by-metric analysis of your synthetic data.
 - More in-depth evaluation.
- Quality test:
 - Quality report (suggested): statistical similarity between real data and synthetic data. If the synthetic and real data are statistically similar, we refer to the synthetic data as being high quality.
 - CategoryCoverage: whether a synthetic column covers all the possible categories that are present in a real column.
 - RangeCoverage: whether a synthetic column covers the full range of values that are present in a real column. It is only for continuous values.
- Privacy test:
 - DCRBaselineProtection:
 - Measures the distance between your synthetic data and real data to measure how private it is.
 - DCROverfittingProtection:
 - Measures the distance between your synthetic data and real training data to measure how private it is.
 - You must have a separate dataset that has not been used for training.

HOW TO EVALUATE: SDMETRICS

- CategoryCoverage:

```
from sdmetrics.single_column import CategoryCoverage

CategoryCoverage.compute(
    real_data=real_table['column_name'],
    synthetic_data=synthetic_table['column_name']
)
```

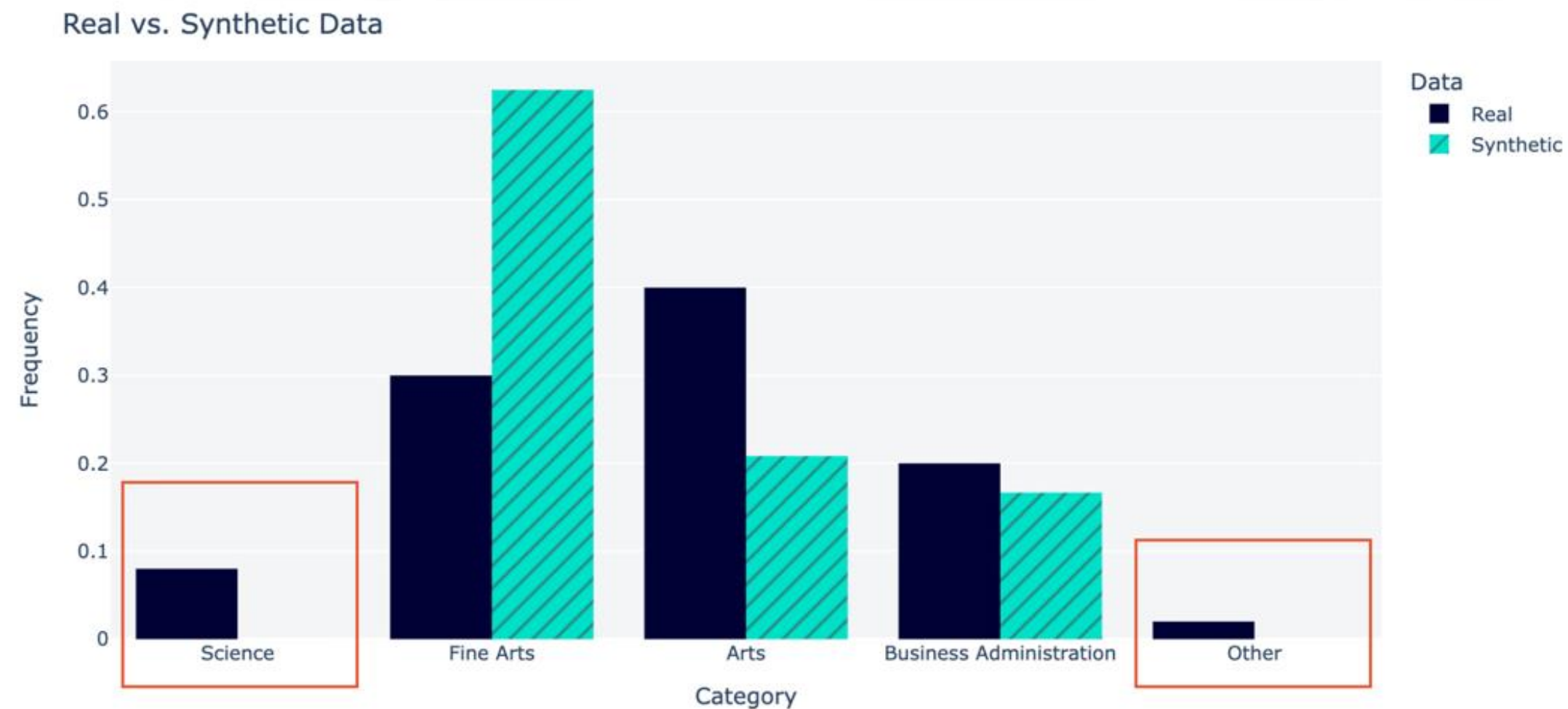
- Range Coverage:

```
from sdmetrics.single_column import RangeCoverage

RangeCoverage.compute(
    real_data=real_table['column_name'],
    synthetic_data=synthetic_table['column_name']
)
```

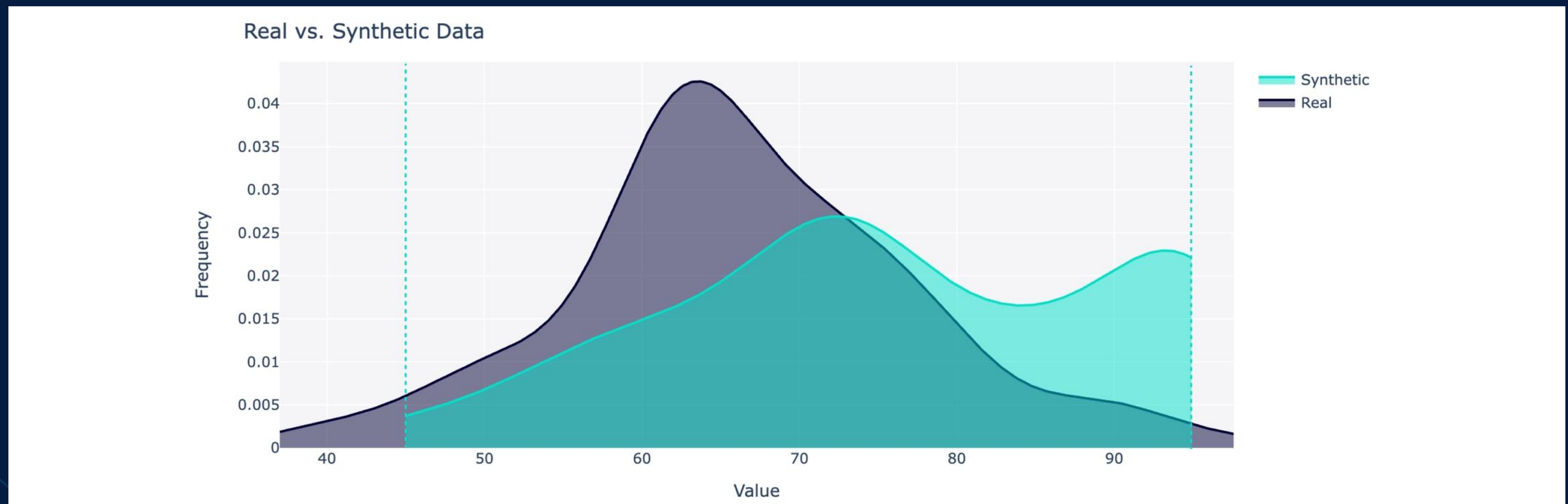

HOW TO EVALUATE: SDMETRICS

- CategoryCoverage:
 - Example:
 - Model fails to generate data for two of 5 categories (Score: 60%).



HOW TO EVALUATE: SDMETRICS

- RangeCoverage:
 - Example:
 - Real data is in range $[37.0, 97.7]$ while the synthetic data is in the range $[45, 95]$. The synthetic data fails to cover the lower end and higher ends of the real values: $[37, 45]$ and $[95, 97.7]$. The missing ranges account for roughly 18% of the overall range, making the RangeCoverage 0.82.



HOW TO EVALUATE: SDMETRICS

- DCRBaselineProtection:
 - Example

```
{
  'score': 0.58808,
  'median_DCR_to_real_data': {
    'synthetic_data': 0.12355,
    'random_data_baseline': 0.21009
  }
}
```

- DCROverfittingProtection
 - Example:

```
{
  'score': 0.5,
  'synthetic_data_percentages': {
    'closer_to_training': 0.75,
    'closer_to_holdout': 0.25
  }
}
```

Resources

- [Welcome to the SDV! | Synthetic Data Vault](#)
- [SDMetrics | SDMetrics](#)
- [SDV.pdf](#)



ARC Training Centre for Behavioural
Insights for Technology Adoption

Thank you

www.arcbita.org

